



BURSA TEKNİK ÜNİVERSİTESİ

BURSA TEKNİK ÜNİVERSİTESİ BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ 2025-2026 GÜZ DÖNEMİ ALGORİTMALAR VE PROGRAMLAMA DERSİ DÖNEM PROJESİ

AD SOYAD: Elif ÇİN

NUMARA: 25360859042

ŞUBE: 1. Şube

BÖLÜM: Bilgisayar Mühendisliği

PROJE KONUSU: Konsol Tabanlı Fizik Simülasyonu

GITHUB: [https://github.com/Nurr-
E/BLM101_25360859042_Elif-in](https://github.com/Nurr-E/BLM101_25360859042_Elif-in)

Bu proje bireysel olarak geliştirilmiştir.

1. GİRİŞ

Bu proje, Bursa Teknik Üniversitesi Bilgisayar Mühendisliği Bölümü Algoritmalar ve Programlama dersi dönem ödevi kapsamında, C programlama dili kullanılarak geliştirilmiş konsol tabanlı bir uzay simülasyonu uygulamasıdır. Uygulama, herhangi bir grafiksel arayüz içermemekte olup, tüm kullanıcı etkileşimi ve çıktıları metin tabanlı olarak konsol ekranı üzerinden sağlanmaktadır. Projenin temel amacı, bir bilim insanının uzay ortamında temel fizik kurallarını (serbest düşme, hidrostatik basınç, sarkaç hareketi vb.) simüle etmesini sağlamak ve bu deneylerin sonuçlarını Güneş Sistemi'ndeki farklı gezegen koşullarında karşılaştırmalı olarak sunmaktır.

Programın genel çalışma akışı şu şekildedir: Simülasyon, kullanıcının (bilim insanının) adının sisteme girilmesiyle başlar. Ardından kullanıcıya simülasyon kapsamında yapılabilecek deneylerin listesini içeren bir seçim menüsü sunulur. Kullanıcı bu menüden bir deney seçtiğinde, program ilgili deneyin hesaplanabilmesi için gerekli fiziksel metrikleri (kütle, uzunluk, süre vb.) girmesini ister. Girilen veriler doğrultusunda, seçilen fiziksel olayın sonucu, Güneş Sistemi'nde yer alan tüm gezegenlerin yerçekimi ivmeleri kullanılarak ayrı ayrı hesaplanır ve elde edilen sonuçlar ilgili birimleriyle birlikte ekrana alt alta yazdırılır.

2. TEKNİK DETAYLAR

Bu bölümde, geliştirilen uzay simülasyonu projesinin kod mimarisi, veri yapıları, fiziksel hesaplama algoritmaları ve hata yönetimi mekanizmaları detaylandırılmıştır. Proje, C programlama dili kullanılarak modüler bir yapıda tasarlanmış olup, bellek yönetimi ve işaretçi (pointer) aritmetiği kurallarına sıkı sıkıya bağlı kalınmıştır.

2.1. Program Akışı ve Modüler Yapı

Program, "böl ve yönet" prensibine dayalı modüler bir mimariye sahiptir. Her bir fizik deneyi, ana akıştan bağımsız çalışan ayrı bir void fonksiyon olarak tanımlanmıştır. Programın giriş noktası olan main fonksiyonu, simülasyonun yönetim merkezi olarak görev yapar.

Simülasyon başladığında ilk olarak kullanıcıdan bilim insanı adı istenir ve bu isim tüm deney süreci boyunca kullanılır. Ardından, kullanıcı -1 değerini girerek çıkış yapana kadar aktif kalan

bir döngü (loop) içerisinde ana menü görüntülenir. Kullanıcının menüden yaptığı seçimler switch-case yapısı ile kontrol edilerek ilgili deney fonksiyonuna yönlendirme yapılır. Tüm deney fonksiyonlarına, gezegen verilerini içeren dizi, bellek verimliliği ve proje isterleri doğrultusunda pointer (işaretçi) olarak parametre geçilir.

```

// Gezegen isimleri
const char *GEZEGENLER[] = {
    "Merkur", "Venus", "Dunya", "Mars",
    "Jupiter", "Saturn", "Uranus", "Neptun"
};

// Fonksiyon Prototipleri
void serbest_dusme(double *g_ptr);
void yukari_atis(double *g_ptr);
void agirlik_deneyi(double *g_ptr);
void potansiyel_enerji(double *g_ptr);
void hidrostatik_basinc(double *g_ptr);
void arsimet_kaldirma(double *g_ptr);
void sarka_periyodu(double *g_ptr);
void ip_gerilmesi(double *g_ptr);
void asansor_deneyi(double *g_ptr);
void menu_goster();

int main() {

    char bilim_insani[100];
    printf("--- UZAY SIMULASYONUNA HOS GELDINIZ ---\n");
    printf("Lutfen Bilim Insani Adini ve Soyadini Giriniz: ");
    fgets(bilim_insani, sizeof(bilim_insani), stdin);

    // Merkür, Venüs, Dünya, Mars, Jüpiter, Satürn, Uranüs, Neptün
    double g_values[] = {3.7, 8.87, 9.807, 3.71, 24.79, 10.44, 8.69, 11.15};
    double *g_ptr = g_values;

    int secim = 0;

    while (secim != -1) {
        printf("\n===== \n");
        printf("Bilim Insani: %s\n", bilim_insani);
        menu_goster();
        printf("Seciminiz (Cikis icin -1): ");
        scanf("%d", &secim);
        switch (secim) {
            case 1: serbest_dusme(g_ptr); break;
            case 2: yukari_atis(g_ptr); break;
            case 3: agirlik_deneyi(g_ptr); break;
            case 4: potansiyel_enerji(g_ptr); break;
            case 5: hidrostatik_basinc(g_ptr); break;
            case 6: arsimet_kaldirma(g_ptr); break;
            case 7: sarka_periyodu(g_ptr); break;
            case 8: ip_gerilmesi(g_ptr); break;
            case 9: asansor_deneyi(g_ptr); break;
            case -1:
                printf("Simulasyon sonlandiriliyor... Iyi calismalar %s.\n", bilim_insani);
                break;
            default:
                printf("Gecersiz secim! Lutfen tekrar deneyiniz.\n");
        }
    }

    return 0;
}

```

ŞEKİL 1.1: Programın ana kontrol döngüsü, main fonksiyonu ve prototipleri gösteren kod görüntüsü

```
=====
Bilim İnsanı: Elif ÇİN

-----
1. Serbest Düşme Deneyi
2. Yukarı Atış Deneyi
3. Ağırlık Deneyi
4. Kutleçekimsel Potansiyel Enerji Deneyi
5. Hidrostatik Basıncı Deneyi
6. Arsimet Kaldırma Kuvveti Deneyi
7. Basit Sarkaç Periyodu Deneyi
8. Sabit İp Gerilmesi Deneyi
9. Asansör Deneyi
-----
Seçiminiz (Çıkış için -1): █
```

ŞEKİL 1.2: Programın açılış ekranı ve ana menü yapısı

2.2. Gezegen Verileri ve Kullanılan Sabitler

Projede, Güneş Sistemi'nde yer alan 8 gezegenin yüzey yerçekimi ivmeleri (m/s^2) double veri tipinde bir dizi içerisinde tutulmaktadır. Merkür'den başlayarak Neptün'e kadar sıralanan bu dizinin yönetimi tamamen pointer mantığıyla sağlanmıştır.

- **Dizi Yapısı:** `double g_values[] = {3.7, 8.87, 9.807, ...}` şeklinde tanımlanmıştır.
- **Erişim Yöntemi:** Dizi elemanlarına `dizi[i]` operatörü yerine, projenin teknik zorunluluğu olan `*(g_ptr + i)` şeklinde pointer aritmetiği kullanılarak erişilmektedir.
- **Sabitler:** Hesaplamalarda hassasiyeti sağlamak amacıyla PI sayısı ve birim dönüşümleri standart fizik kurallarına göre tanımlanmıştır.

2.3. Deneylerin Hesaplama Mantığı

2.3.1. Serbest Düşme Deneyi

Serbest düşme deneyi simülasyonu, hava direncinin ve sürtünme kuvvetinin ihmal edildiği ideal bir ortam varsayımı üzerine kurulmuştur. Bu deneyde, kullanıcıdan cismin serbest bırakıldığı andan itibaren geçen süreyi (t) saniye cinsinden girmesi beklenir. Program, arka planda bu süreyi parametre olarak alır ve `const double` dizisi içerisinde saklanan gezegenlerin yerçekimi ivmelerine (g) işaretçi (pointer) aritmetiği ile sırayla erişim sağlar. Şekil 2.1'de gösterilen kod bloğunda görüleceği üzere, hesaplama aşamasında $h = 1/2gt^2$ formülü uygulanarak, cismin her bir gezegenin yüzeyinde o süre zarfında kat edeceği dikey mesafe (h) bulunur. Kod içerisinde süre değişkeninin karesi alınarak ilgili gezegenin ivme değeriyle çarpılır ve sonuç ikiye bölünür. Elde edilen düşüş mesafesi değerleri, simülasyonun doğruluğu açısından metre (m) biriminde Şekil 2.2'deki gibi kullanıcıya sunulur.

```
// 1. Serbest Düşme Deneyi
void serbest_dusme(double *g_ptr) {
    double t, h;
    printf("\n--- Serbest Dusme Deneyi ---\n");
    printf("Dusus suresini giriniz (sn): ");
    scanf("%lf", &t);

    t = (t < 0) ? -t : t;
    if(t != (t < 0 ? -t : t)) printf("Negatif deger mutlak degere cevrildi: %.2f\n", t);

    printf("\n%-10s | %-15s\n", "GEZEGEN", "YOL (h) [m]");
    printf("-----\n");

    for (int i = 0; i < 8; i++) {
        double g = *(g_ptr + i);
        h = 0.5 * g * t * t;
        printf("%-10s | %.2f m\n", GEZEGENLER[i], h);
    }
}
```

Şekil 2.1: Serbest düşme deneyi kod bloğu

```
Seciminiz (Cikis icin -1): 1

--- Serbest Dusme Deneyi ---
Dusus suresini giriniz (sn): 10

GEZEGEN      | YOL (h) [m]
-----
Merkur       | 185.00 m
Venus        | 443.50 m
Dunya        | 490.35 m
Mars         | 185.50 m
Jupiter      | 1239.50 m
Saturn       | 522.00 m
Uranus       | 434.50 m
Neptun       | 557.50 m
```

Şekil 2.2: Serbest düşme deneyi ekran çıktısı

2.3.2. Yukarı Atış Deneyi

Bu deney modülünde, bir cismin yerçekimine karşı dikey doğrultuda fırlatılması durumu simüle edilmektedir. Kullanıcıdan sistem girdisi olarak cismin ilk fırlatılma hızı (V_0) metre/saniye cinsinden talep edilir. Fiziksel olarak cismin kinetik enerjisinin tamamen potansiyel enerjiye dönüştüğü tepe noktasını bulmak için $h(\max) = (V_0)^2/2g$ formülü, Şekil 2.3'te yer alan algoritma içerisinde işlenir. Kod yapısı, girilen hız değerinin karesini alarak, işaretçi aracılığıyla erişilen yerçekimi ivmesinin iki katına böler. Yerçekimi ivmesi ile çıkılabilecek maksimum yükseklik ters orantılı olduğundan, algoritma Merkür gibi düşük

yerçekimli gezegenlerde çok daha yüksek $h(\max)$ değerleri üretirken, Jüpiter gibi dev gezegenlerde bu değer oldukça düşük çıktığını simüle eder. Hesaplanan sonuçlar Şekil 2.4'te görüldüğü üzere metre (m) hassasiyetinde ekrana yansıtılır.

```
// 2. Yukarı Atış Deneyi
void yukari_atis(double *g_ptr) {
    double v0, h_max;
    printf("\n--- Yukari Atis Deneyi ---\n");
    printf("Firlatma hizini giriniz (m/s): ");
    scanf("%lf", &v0);

    v0 = (v0 < 0) ? -v0 : v0;

    printf("\n%-10s | %-15s\n", "GEZEGEN", "MAX YUKSEKLIK (m)");
    printf("-----\n");

    for (int i = 0; i < 8; i++) {
        double g = *(g_ptr + i);
        h_max = (v0 * v0) / (2 * g);
        printf("%-10s | %.2f m\n", GEZEGENLER[i], h_max);
    }
}
```

Şekil 2.3: Yukarı atış deneyi kod bloğu

```
Seciminiz (Cikis icin -1): 2
--- Yukari Atis Deneyi ---
Firlatma hizini giriniz (m/s): 25

GEZEGEN      | MAX YUKSEKLIK (m)
-----
Merkur       | 84.46 m
Venus        | 35.23 m
Dunya        | 31.86 m
Mars         | 84.23 m
Jupiter      | 12.61 m
Saturn       | 29.93 m
Uranus       | 35.96 m
Neptun       | 28.03 m
```

Şekil 2.4: Yukarı atış deneyi ekran çıktısı

2.3.3. Ağırlık Deneyi

Kütle ve ağırlık kavramları arasındaki temel farkın simülasyon ortamında gösterilmesi amacıyla tasarlanan bu deneyde, kullanıcıdan değişmeyen madde miktarı olan kütle (m) bilgisini kilogram (kg) cinsinden girmesi istenir. Program, girilen skaler kütle değerini, bellekte dizi halinde tutulan vektörel yerçekimi ivmeleri (g) ile çarparak ağırlığı hesaplar. Bu işlem için Şekil 2.5'teki kod yapısında $G = mg$ formülü kullanılır. Kod bloğu, bir döngü içerisinde gezegen dizisinin adreslerinde gezerek her gezegen için özelleşmiş ağırlık değerini hesaplar. Elde edilen sonuçlar, kuvvet birimi olan Newton (N) cinsinden Şekil 2.6'daki listede gösterilerek,

kullanıcının aynı kütlenin farklı gök cisimlerinde ne kadar farklı ağırlıklara karşılık geldiğini gözlemlemesi sağlanır.

```
// 3. Ağırlık Deneyi
void agirlik_deneyi(double *g_ptr) {
    double m, G;
    printf("\n--- Agirlik Deneyi ---\n");
    printf("Cismin kutlesini giriniz (kg): ");
    scanf("%lf", &m);

    m = (m < 0) ? -m : m;

    printf("\n%-10s | %-15s\n", "GEZEGEN", "AGIRLIK (N)");
    printf("-----\n");

    for (int i = 0; i < 8; i++) {
        double g = *(g_ptr + i);
        G = m * g;
        printf("%-10s | %.2f N\n", GEZEGENLER[i], G);
    }
}
```

Şekil 2.5: Ağırlık deneyi kod bloğu

```
Seciminiz (Cikis icin -1): 3

--- Agirlik Deneyi ---
Cismin kutlesini giriniz (kg): 50

GEZEGEN      | AGIRLIK (N)
-----
Merkur       | 185.00 N
Venus        | 443.50 N
Dunya        | 490.35 N
Mars          | 185.50 N
Jupiter      | 1239.50 N
Saturn        | 522.00 N
Uranus       | 434.50 N
Neptun       | 557.50 N
```

Şekil 2.6: Ağırlık deneyi ekran çıktısı

2.3.4. Kütleçekimsel Potansiyel Enerji Deneyi

Cismin bulunduğu konum itibarıyla sahip olduğu enerjiyi modelleyen bu deneyde, algoritma iki farklı değişken talep eder: cismin kütlesi (m) ve bulunduğu yükseklik (h). Program, alınan bu verileri kullanarak her gezegen için potansiyel enerjiyi $E_p = mgh$ formülü ile hesaplar. Şekil 2.7'de sunulan kodsalları açıdan bu modül, üç farklı double değişkenin çarpımını içerir ve gezegenlerin g değerlerine pointer ile ulaşarak işlemi gerçekleştirir. Enerji, skaler bir büyüklük olup yerçekimi ivmesi ile doğru orantılıdır. Simülasyon sonucunda, girilen kütle ve yükseklik

değerlerine karşılık gelen enerji miktarı Joule (J) birimiyle, Şekil 2.8'de olduğu gibi bilimsel gösterim standartlarına uygun olarak ekrana yazdırılır.

```
// 4. Potansiyel Enerji Deneyi
void potansiyel_enerji(double *g_ptr) {
    double m, h, Ep;
    printf("\n--- Potansiyel Enerji Deneyi ---\n");
    printf("Cismin kutlesini giriniz (kg): ");
    scanf("%lf", &m);
    printf("Yuksekligi giriniz (m): ");
    scanf("%lf", &h);

    m = (m < 0) ? -m : m;
    h = (h < 0) ? -h : h;

    printf("\n%-10s | %-15s\n", "GEZEGEN", "ENERJİ (Joule)");
    printf("-----\n");

    for (int i = 0; i < 8; i++) {
        double g = *(g_ptr + i);
        Ep = m * g * h;
        printf("%-10s | %.2f J\n", GEZEGENLER[i], Ep);
    }
}
```

Şekil 2.7: Potansiyel enerji deneyi kod bloğu

```
--- Potansiyel Enerji Deneyi ---
Cismin kutlesini giriniz (kg): 20
Yuksekligi giriniz (m): 45

GEZEGEN      | ENERJİ (Joule)
-----
Merkur       | 3330.00 J
Venus        | 7983.00 J
Dunya        | 8826.30 J
Mars         | 3339.00 J
Jupiter      | 22311.00 J
Saturn       | 9396.00 J
Uranus       | 7821.00 J
Neptun       | 10035.00 J
```

Şekil 2.8: Potansiyel enerji deneyi ekran çıktısı

2.3.5. Hidrostatik Basınç Deneyi

Sıvı mekaniği prensiplerinin uzay ortamına uyarlandığı bu deneyde, durgun bir sıvının belirli bir derinlikte oluşturduğu basınç farkları incelenir. Kullanıcıdan sıvının yoğunluğu (ρ) kg/m^3 ve derinlik (h) metre cinsinden istenir. Program, bu girdileri Şekil 2.9'daki fonksiyonda $P = \rho gh$ formülünde yerine koyarak hesaplama yapar. Algoritma, yerçekimi ivmesinin sıvı basıncı üzerindeki doğrudan etkisini simüle eder; bu sayede aynı derinlikteki aynı sıvının, farklı gezegenlerde yüzeye uyguladığı basınç kuvvetinin değişimi gözlemlenir. Hesaplamalar sonucunda elde edilen veriler, basınç birimi olan Pascal (Pa) cinsinden hassas bir şekilde (ondalık basamaklı) Şekil 2.10'daki kullanıcı arayüzüne aktarılır.

```
// 5. Hidrostatik Basınc Deneyi
void hidrostatik_basinc(double *g_ptr) {
    double rho, h, P;
    printf("\n--- Hidrostatik Basınc Deneyi ---\n");
    printf("Sıvının yoğunlugunu giriniz (kg/m3): ");
    scanf("%lf", &rho);
    printf("Derinligi giriniz (m): ");
    scanf("%lf", &h);

    rho = (rho < 0) ? -rho : rho;
    h = (h < 0) ? -h : h;

    printf("\n%-10s | %-15s\n", "GEZEGEN", "BASINC (Pascal)");
    printf("-----\n");

    for (int i = 0; i < 8; i++) {
        double g = *(g_ptr + i);
        P = rho * g * h;
        printf("%-10s | %.2f Pa\n", GEZEGENLER[i], P);
    }
}
```

Şekil 2.9: Hidrostatik basınç deneyi kod bloğu

```
--- Hidrostatik Basınc Deneyi ---
Sıvının yoğunlugunu giriniz (kg/m3): 15
Derinligi giriniz (m): 20

GEZEGEN      | BASINC (Pascal)
-----
Merkur       | 1110.00 Pa
Venus        | 2661.00 Pa
Dunya        | 2942.10 Pa
Mars         | 1113.00 Pa
Jupiter      | 7437.00 Pa
Saturn       | 3132.00 Pa
Uranus       | 2607.00 Pa
Neptun       | 3345.00 Pa
```

Şekil 2.10: Hidrostatik basınç deneyi ekran çıktısı

2.3.6. Arşimet Kaldırma Kuvveti Deneyi

Sıvı içerisinde bırakılan bir cisme etki eden yukarı yönlü kaldırma kuvvetinin simülasyonu, Arşimet prensibine dayanır. Bu modülde kullanıcıdan sıvının yoğunluğu (ρ) ve cismin batan hacmi (V) parametreleri talep edilir. Hesaplama çekirdeğinde, Şekil 2.11'de gösterildiği üzere $F_k = \rho g V$ formülü kullanılır. Kod yapısı, sıvının yoğunluğu ve batan hacim sabit kalmak kaydıyla, kaldırma kuvvetinin gezegenin yerçekimi ivmesine (g) bağlı olarak nasıl değiştiğini analiz eder. Pointer aritmetiği ile çekilen her bir g değeri formülde işlenir ve sonuç kuvvet birimi olan Newton (N) olarak üretilir. Bu deney, Şekil 2.12'deki çıktılarda görüldüğü gibi, kaldırma kuvvetinin sadece sıvıya değil, ortamın çekim alanına da bağlı olduğunu teknik olarak kanıtlar niteliktedir.

```
// 6. Arşimet Kaldırma Kuvveti
void arsimet_kaldirma(double *g_ptr) {
    double rho, V, Fk;
    printf("(char [38])"Sivinin yogunlugunu giriniz (kg/m3): ");
    printf("Sivinin yogunlugunu giriniz (kg/m3): ");
    scanf("%lf", &rho);
    printf("Cismin batan hacmini giriniz (m3): ");
    scanf("%lf", &V);

    rho = (rho < 0) ? -rho : rho;
    V = (V < 0) ? -V : V;

    printf("\n%-10s | %-15s\n", "GEZEGEN", "KALDIRMA KUV. (N)");
    printf("-----\n");

    for (int i = 0; i < 8; i++) {
        double g = *(g_ptr + i);
        Fk = rho * g * V;
        printf("%-10s | %.2f N\n", GEZEGENLER[i], Fk);
    }
}
```

Şekil 2.11: Arşimet kaldırma kuvveti deneyi kod bloğu

```

--- Arşimet Kaldırma Kuvveti Deneyi ---
Sivinin yoğunlugunu giriniz (kg/m3): 25
Cismin batan hacmini giriniz (m3): 40

```

GEZEGEN	KALDIRMA KUV. (N)
Merkur	3700.00 N
Venus	8870.00 N
Dunya	9807.00 N
Mars	3710.00 N
Jupiter	24790.00 N
Saturn	10440.00 N
Uranus	8690.00 N
Neptun	11150.00 N

Şekil 2.12: Arşimet kaldırma kuvveti deneyi ekran çıktısı

2.3.7. Basit Sarkaç Periyodu Deneyi

Basit harmonik hareket yapan bir sarkacın salınım süresini (periyot) hesaplayan bu deney, özellikle math.h kütüphanesinin kullanımını gerektiren bir yapıya sahiptir. Kullanıcıdan sarkacın ip uzunluğu (L) metre cinsinden istenir. Periyot hesabı için $T = 2\pi\sqrt{L/g}$ formülü kullanılır. Şekil 2.13'teki kod yapısında görüldüğü üzere, C programlama dilinde karekök alma işlemi sqrt() fonksiyonu ile gerçekleştirilirken, pi sayısı yüksek hassasiyetli bir sabit (define veya const) olarak tanımlanmıştır. Formülde yerçekimi ivmesi (g) paydada ve kök içinde yer aldığı için, yerçekiminin arttığı gezegenlerde periyodun kısaldığı, yani sarkacın daha hızlı salındığı simüle edilir. Çıktılar Şekil 2.14'te saniye (s) biriminde verilir.

```

// 7. Basit Sarkaç Periyodu
void sarka_periyodu(double *g_ptr) {
    double L, T;
    printf("\n--- Basit Sarkaç Periyodu Deneyi ---\n");
    printf("Sarkac uzunlugunu giriniz (m): ");
    scanf("%lf", &L);

    L = (L < 0) ? -L : L;

    printf("\n%-10s | %-15s\n", "GEZEGEN", "PERİYOT (sn)");
    printf("-----\n");

    for (int i = 0; i < 8; i++) {
        double g = *(g_ptr + i);
        T = 2 * PI * sqrt(L / g);
        printf("%-10s | %.2f s\n", GEZENLER[i], T);
    }
}

```

Şekil 2.13: Basit sarkaç periyodu deneyi kod bloğu

```

Seciminiz (Cikis icin -1): 7

--- Basit Sarkac Periyodu Deneyi ---
Sarkac uzunlugunu giriniz (m): 80

GEZEGEN      | PERİYOT (sn)
-----
Merkur       | 29.22 s
Venus        | 18.87 s
Dunya        | 17.95 s
Mars         | 29.18 s
Jupiter      | 11.29 s
Saturn       | 17.39 s
Uranus       | 19.06 s
Neptun       | 16.83 s

```

2.14: Basit sarkaç periyodu deneyi ekran çıktısı

2.3.8. Sabit İp Gerilmesi Deneyi

Düşey doğrultuda asılı, hareketsiz bir kütlenin esnemez ve kütsesiz kabul edilen bir ipten oluşturduğu gerilme kuvveti bu deneyde hesaplanır. Kullanıcıdan sadece asılan cismin kütlesi (m) istenir. Fiziksel olarak sistem dengede olduğu için ipteki gerilme kuvveti cismin ağırlığına eşittir ve Şekil 2.15'te uygulandığı üzere $T = mg$ formülü ile modellenir. Ağırlık deneyi ile matematiksel formülasyonu aynı olsa da, bu deneyin fiziksel bağlamı "gerilme kuvveti" üzerine kuruludur. Algoritma, girilen kütleyi her gezegenin g değeriyle işleyerek ipten oluşacak gerilmeyi Newton (N) cinsinden hesaplar ve Şekil 2.16'daki gibi listeler.

```

// 8. Sabit İp Gerilmesi
void ip_girilmesi(double *g_ptr) {
    double m, T;
    printf("\n--- Sabit İp Gerilmesi Deneyi ---\n");
    printf("Cismin kutlesini giriniz (kg): ");
    scanf("%lf", &m);

    m = (m < 0) ? -m : m;

    printf("\n%-10s | %-15s\n", "GEZEĞEN", "GERİLME (N)");
    printf("-----\n");

    for (int i = 0; i < 8; i++) {
        double g = *(g_ptr + i);
        T = m * g;
        printf("%-10s | %.2f N\n", GEZEĞENLER[i], T);
    }
}

```

Şekil 2.15: Sabit ip gerilmesi deneyi kod bloğu

```
Seciminiz (Cikis icin -1): 8

--- Sabit Ip Gerilmesi Deneyi ---
Cismin kutlesini giriniz (kg): 10

GEZEGEN      | GERILME (N)
-----
Merkur       | 37.00 N
Venus        | 88.70 N
Dunya        | 98.07 N
Mars         | 37.10 N
Jupiter      | 247.90 N
Saturn       | 104.40 N
Uranus       | 86.90 N
Neptun       | 111.50 N
```

Şekil 2.16: Sabit ip gerilmesi deneyi ekran çıktısı

2.3.9. Asansör Deneyi

Eylemsiz referans sistemlerinde, ivmeli hareketin cisim üzerindeki etkisini gösteren bu deney en kapsamlı hesaplama mantığına sahiptir. Kullanıcıdan cismin kütlesi (m), asansörün ivmesi (a) ve hareketin yönü (yukarı/aşağı hızlanma-yavaşlama durumu) parametreleri istenir. Algoritma, Şekil 2.17'de görüldüğü gibi kullanıcının seçimine göre bir if-else karar yapısı kullanır. Eğer asansör yukarı yönde hızlanıyorsa (veya aşağı yavaşlıyorsa) cismin üzerindeki etkin kuvvet artacağından $N = m(g+a)$ formülü; aşağı yönde hızlanıyorsa (veya yukarı yavaşlıyorsa) üzerindeki etkin kuvvet azalacağından $N = m(g-a)$ formülü devreye girer. Bu modül, Newton'un hareket yasalarının simülasyon ortamında dinamik olarak işlenmesini sağlar ve sonuçları Şekil 2.18'de Newton (N) biriminde üretir.

```
// 9. Asansör Deneyi
void asansor_deneyi(double *g_ptr) {
    double m, a, N;
    int yon_secimi;
    printf("\n--- Asansor Deneyi ---\n");
    printf("1. Asansor YUKARI ivmelenerek hizlaniyor VEYA ASAGI ivmelenerek yavasliyor (N = m(g+a))\n");
    printf("2. Asansor ASAGI ivmelenerek hizlaniyor VEYA YUKARI ivmelenerek yavasliyor (N = m(g-a))\n");
    printf("Durum seciniz (1 veya 2): ");
    scanf("%d", &yon_secimi);
    printf("Cismin kutlesini giriniz (kg): ");
    scanf("%lf", &m);
    printf("Asansor ivmesini giriniz (m/s^2): ");
    scanf("%lf", &a);

    m = (m < 0) ? -m : m;
    a = (a < 0) ? -a : a;

    printf("\n%-10s | %-15s\n", "GEZEGEN", "ETKIN AGIRLIK (N)");
    printf("-----\n");

    for (int i = 0; i < 8; i++) {
        double g = *(g_ptr + i);

        if (yon_secimi == 1) {
            N = m * (g + a);
        } else {
            N = m * (g - a);

            if (N < 0) N = 0;
        }
        printf("%-10s | %.2f N\n", GEZEGENLER[i], N);
    }
}
```

Şekil 2.17: Asansör deneyi kod bloğu

```
2. Asansor ASAGI ivmelenerek hizlaniyor VEYA YUKARI ivmelenerek yavasliyor (N = m(g-a))
Durum seciniz (1 veya 2): 1
Cismin kutlesini giriniz (kg): 10
Asansor ivmesini giriniz (m/s^2): 5
```

GEZEGEN	ETKIN AGIRLIK (N)
Merkur	87.00 N
Venus	138.70 N
Dunya	148.07 N
Mars	87.10 N
Jupiter	297.90 N
Saturn	154.40 N
Uranus	136.90 N
Neptun	161.50 N

Şekil 2.18: Asansör deneyi ekran çıktısı

3. EKSİKLİKLER VE GELİŞTİRMELER

Bu proje, Algoritmalar ve Programlama dersi dönem ödevi kapsamında belirlenen isterleri (konsol tabanlı yapı, pointer aritmetiği kullanımı, modüler fonksiyon yapısı vb.) tam olarak

karşılayan çalışan bir prototiptir. Ancak, yazılım geliştirme döngüsünün doğası gereği, her projenin geliştirilmeye açık yönleri bulunmaktadır. Projenin mevcut sürümündeki kısıtlar ve sonraki versiyonlarda sisteme entegre edilmesi planlanan teknik ve fonksiyonel geliştirmeler aşağıda maddeler halinde sunulmuştur.

3.1. Veri Kalıcılığı ve Dosya Yönetimi (File I/O)

Mevcut simülasyon, verileri yalnızca programın çalışma süresi boyunca (Runtime) bellekte tutmakta ve sonuçları anlık olarak konsol ekranına yazdırmaktadır. Program kapatıldığında yapılan deneyler ve elde edilen hesaplamalar kaybolmaktadır.

Planlanan Geliştirme: C programlama dilinin dosya işleme yetenekleri (fopen, fprintf vb.) kullanılarak, bilim insanının gerçekleştirdiği her deneyin sonucunun tarih ve saat etiketiyle birlikte bir .txt veya veri analizine uygun .csv dosyasına kaydedilmesi sağlanabilir. Böylece kullanıcı, geçmiş deney verilerine erişim sağlayabilir ve uzun vadeli veri analizi yapabilir.

3.2. Fiziksel Modelleme Gerçekçiliği (Atmosfer Etkisi)

Simülasyonda kullanılan fiziksel formüller (örneğin serbest düşme için $h=1/2gt^2$), sürtünmesiz ideal ortam varsayımına dayanmaktadır. Ancak gerçek uzay koşullarında, özellikle Venüs veya Mars gibi gezegenlerde atmosfer yoğunluğu, hareket eden cisimler üzerinde ciddi bir aerodinamik direnç oluşturur.

Planlanan Geliştirme: Simülasyonun fizik motoruna "Hava Sürtünmesi" ve "Limit Hız" kavramları eklenebilir. Her gezegen için tanımlanacak bir atmosferik yoğunluk katsayısı ile cisimlerin düşüş süreleri daha gerçekçi bir şekilde modellenenebilir.

3.3. Veri Yapılarının Optimizasyonu (Struct Kullanımı)

Projenin şu anki mimarisinde, gezegen isimleri ve yerçekimi ivmeleri birbirine paralel iki farklı dizi (array) üzerinde tutulmaktadır. Bu yapı, veri sayısı arttığında yönetimi zorlaştırabilir.

Planlanan Geliştirme: Gezegenlere ait tüm özelliklerin (isim, yerçekimi ivmesi, atmosfer durumu, yarıçap vb.) tek bir çatı altında toplandığı struct Gezegen (Yapı) veri modeli oluşturulabilir. Bu sayede veriler daha organize tutulabilir ve nesne yönelimli programlamaya (OOP) geçiş için bir zemin hazırlanabilir.

3.4. Dinamik Bellek Yönetimi ve Gezegen Ekleme

Mevcut kodda gezegen sayısı ve özellikleri kaynak kod içerisinde sabit (const) olarak tanımlanmıştır ve derleme sonrası değiştirilemez.

Planlanan Geliştirme: Dinamik bellek yönetimi (malloc, realloc) kullanılarak, kullanıcının program çalışırken kendi keşfettiği veya kurguladığı yeni bir gezegeni sisteme eklemesine olanak tanınabilir. Kullanıcı, "Yeni Gezegen Ekle" menüsü üzerinden gezegen adını ve yerçekimi ivmesini girerek simülasyon evrenini genişletebilir.

3.5. Görselleştirme (Grafik Arayüz)

Proje, isterler gereği metin tabanlı (Text-Based) bir arayüz ile sınırlandırılmıştır. Ancak fiziksel olayların sadece sayılarla ifade edilmesi, kullanıcının olayı zihninde canlandırmasını zorlaştırabilir.

Planlanan Geliştirme: İlerleyen aşamalarda basit bir grafik kütüphanesi entegre edilerek, sarkaç salınımı veya cismin düşüş hareketi 2 boyutlu grafiklerle görselleştirilebilir. Bu, uygulamanın eğitim materyali olarak kullanılabilirliğini artıracaktır.

4. SONUÇ

Bu proje kapsamında, C programlama dilinin sunduğu güçlü bellek yönetimi ve yapısal programlama yetenekleri kullanılarak, modüler mimariye sahip kapsamlı bir konsol tabanlı uzay simülasyonu geliştirilmiştir. Çalışma boyunca, sadece temel fizik kurallarının (Newton mekaniği, akışkanlar dinamiği vb.) algoritmik olarak modellenmesi değil; aynı zamanda **işaretçi (pointer) aritmetiği, dizi-pointer ilişkisi ve ternary operatör kullanımı** gibi teknik konularda derinlemesine yetkinlik kazanılmıştır.

Projenin öne çıkan en önemli yönü, kullanıcı etkileşimini ve veri güvenliğini ön planda tutan sağlam hata yönetimi mekanizmasıdır. Fiziksel büyüklüklerin negatif girilmesini engelleyen kontroller ve dinamik menü yapısı, simülasyonun kararlılığını artırmıştır. Ayrıca, her bir deneyin bağımsız fonksiyonlar halinde tasarlanması, kodun okunabilirliğini ve geliştirilebilirliğini üst seviyeye taşımıştır.

Sonuç olarak; bu dönem projesi sayesinde, teorik fizik bilgileri ile yazılım mühendisliği prensipleri başarılı bir şekilde harmanlanmış, algoritmik düşünme ve problem çözme becerileri pratik uygulamalarla pekiştirilmiştir. Geliştirilen simülasyon, temel bilimsel hesaplamaların bilgisayar ortamında nasıl simüle edilebileceğine dair başarılı bir örnek teşkil etmektedir.

5. KAYNAKÇA

- [1] Bursa Teknik Üniversitesi. (2025). Algoritmalar ve Programlama Dersi Dönem Projesi Dokümanı. Mühendislik ve Doğa Bilimleri Fakültesi, Bilgisayar Mühendisliği Bölümü.
- [2] Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2. Baskı). Prentice Hall. (Not: C dilinin standart kütüphaneleri ve pointer yapısı için referans alınmıştır.)
- [3] Serway, R. A., & Jewett, J. W. (2018). Physics for Scientists and Engineers (Fen ve Mühendislik İçin Fizik). Cengage Learning. (Not: Serbest düşme, sarkaç ve hidrostatik basınç formüllerinin teorik altyapısı için kullanılmıştır.)
- [4] NASA. (2024). Planetary Fact Sheet - Ratio to Earth Values. NASA Goddard Space Flight Center. Erişim Adresi: <https://nssdc.gsfc.nasa.gov/planetary/factsheet/> (Erişim Tarihi: Ocak 2026).
- [5] GeeksforGeeks. (2024). C Pointers and Arrays. Erişim Adresi: <https://www.geeksforgeeks.org/c-pointers/> (Not: Dizi ve pointer aritmetiği arasındaki ilişkiyi incelemek için kullanılmıştır.)